

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 6

Student Name: K.Vinay

Roll No.: A24126552086

Date: 04-10-2026

1. TITLE

Advanced Node.js and Express Concepts

2. AIM

To explore and implement JavaScript classes, built-in Node.js modules, external packages via NPM, and Express.js middleware components.

3. OBJECTIVE

The objectives of this experiment are:

- To distinguish between JavaScript functions and ES6 classes.
- To utilize built-in Node.js modules like os, path, and fs.
- To understand the role of NPM and package.json in managing dependencies.
- To implement and test custom middleware in an Express.js application.

4. THEORY

Node.js extends JavaScript beyond the browser, offering built-in modules like 'os', 'path', and 'fs' to interact natively with the operating system and file system. NPM (Node Package Manager) acts as the ecosystem's registry, allowing developers to install third-party libraries globally or locally. Furthermore, Express.js uses middleware, which are functions that have access to the request and response objects, operating directly in the middle of the HTTP request-response cycle to execute code (such as logging) before routing.

5. ALGORITHM / PROCEDURE

1. Q5: Define a function to calculate area. Define a Student class, instantiate multiple objects, and invoke their methods.
2. Q6: Import 'os', 'path', and 'fs' modules. Print system info, manipulate file paths, and perform synchronous file read/write operations.
3. Q7: Initialize an NPM project. Explain the process of installing external packages and requiring them.
4. Q8: Initialize an Express app. Create a custom requestLogger middleware function to intercept requests and print logs. Define routes and start the server.

6. CODE EXPLANATION

In Q5, the class keyword builds an ES6 blueprint for Student objects. In Q6, standard Node modules are imported with require(). Methods like os.totalmem() and fs.writeFileSync() are executed synchronously. In Q8, express() spins up a server, and app.use(requestLogger) applies a global middleware function. The next() callback within the middleware is explicitly called to pass execution control to subsequent route handlers.

7. PROGRAM

q5_functions_vs_classes.js

```
// Q5: JavaScript Functions vs Classes

// --- 1. FUNCTION EXAMPLE ---
// Functions are reusable blocks of code designed to perform a specific action.
function calculateArea(length, width) {
    return length * width;
}
console.log('Function Output (Area):', calculateArea(5, 10));

// --- 2. CLASS EXAMPLE ---
// A Class is a blueprint (template) for creating multiple objects with shared properties and methods.
class Student {
    // Constructor initializes properties for each object
    constructor(name, rollNo, branch) {
        this.name = name;
        this.rollNo = rollNo;
        this.branch = branch;
    }

    // Common method shared across all instances
    getDetails() {
        return `Student: ${this.name} | Roll No: ${this.rollNo} | Branch: ${this.branch}`;
    }
}

// Creating multiple objects (instances) from the same class blueprint
const student1 = new Student("Alice", 101, "CSE");
const student2 = new Student("Bob", 102, "ECE");
const student3 = new Student("Charlie", 103, "IT");

console.log('\nClass Objects:');
console.log(student1.getDetails());
console.log(student2.getDetails());
console.log(student3.getDetails());
```

q6_nodejs_modules.js

```
// Q6: Node.js Built-in Modules (os, path, fs)

// 1. OS MODULE (Provides operating system information)
const os = require('os');
console.log('=== 1. OS MODULE ===');
console.log('Platform:', os.platform());
```

```

console.log('Architecture:', os.arch());
console.log('Total Memory (GB):', (os.totalmem() / (1024 ** 3)).toFixed(2));
console.log('Free Memory (GB):', (os.freemem() / (1024 ** 3)).toFixed(2));

// 2. PATH MODULE (Handles and transforms file paths across different operating systems)
const path = require('path');
console.log('\n=== 2. PATH MODULE ===');
const samplePath = path.join(__dirname, 'data', 'report.pdf');
console.log('Joined Path:', samplePath);
console.log('File Name:', path.basename(samplePath));
console.log('File Extension:', path.extname(samplePath));

// 3. FS MODULE (Interacts with the file system: create, read, write files)
const fs = require('fs');
console.log('\n=== 3. FS MODULE ===');
const testFile = 'module_demo.txt';
fs.writeFileSync(testFile, 'Hello from Node.js FS module!');
console.log('File written successfully.');
```

```

console.log('File content read:', fs.readFileSync(testFile, 'utf-8'));

// Clean up temporary file
fs.unlinkSync(testFile);

```

q7_npm_and_packages.js

```

// Q7: NPM, package.json, and External Packages in Node.js

/*
=====
EXPLANATION:
1. NPM (Node Package Manager):
    - The default package manager and online registry for JavaScript and Node.js.
    - Used to install, share, and manage open-source third-party libraries.

2. package.json:
    - The configuration and manifest file of a Node.js project.
    - Records metadata (name, version, scripts) and lists all installed dependencies.

=====
STEPS TO INSTALL AND USE AN EXTERNAL PACKAGE:
Step 1: Initialize a Node project (creates package.json)
    Command: npm init -y

Step 2: Install the external package from NPM
    Command: npm install upper-case

Step 3: Import and use the package in your JavaScript file:
=====
*/

// Demonstration of importing an installed external package (or custom module)
console.log('=== STEPS TO USE AN EXTERNAL PACKAGE ===');
console.log('1. Run: npm init -y');
console.log('2. Run: npm install <package-name>');
console.log('3. Use require("package-name") in code.\n');

// Example: Function showing external package behavior

```

```
function convertToUpperCase(str) {
  return str.toUpperCase();
}

console.log('Processed Output:', convertToUpperCase('hello from external package demonstration'));
```

q8_express_middleware.js

```
// Q8: Express.js Middleware & Request Logger

const express = require('express');
const app = express();
const PORT = 3000;

/*
=====
WHAT IS MIDDLEWARE?
Middleware functions are functions that have access to the request object (req),
the response object (res), and the next middleware function (next) in the
application's request-response cycle.

REQUEST-RESPONSE CYCLE:
Client Request ---> [ Middleware 1: Logger ] ---> [ Route Handler ] ---> Client Response
                    | (calls next())
=====
*/

// Custom Logging Middleware
const requestLogger = (req, res, next) => {
  const timestamp = new Date().toLocaleTimeString();
  console.log(`[LOG] ${timestamp} | Method: ${req.method} | URL: ${req.url}`);

  // next() passes control to the next middleware or route handler
  next();
};

// Apply logger middleware to all incoming requests
app.use(requestLogger);

// Sample Routes
app.get('/', (req, res) => {
  res.send('<h2>Home Page</h2><p>Request was successfully logged in terminal!</p>');
});

app.get('/about', (req, res) => {
  res.send('<h2>About Page</h2><p>Middleware handles every incoming route.</p>');
});

// Start Server
app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
  console.log('Try visiting http://localhost:3000/ or http://localhost:3000/about to see logs below:');
});
```

8. TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	node q5_functions_vs_classes.js	Classes output instantiated objects	Successfully logs Student details	Pass
TC-02	node q6_nodejs_modules.js	Modules fetch system info & write file	System architecture & file contents logged correctly	Pass
TC-03	node q7_npm_and_packages.js	NPM external package test	Logs NPM package demonstration successfully	Pass
TC-04	node q8_express_middleware.js	Middleware intercepts traffic	Express server logs incoming requests automatically	Pass

9. OUTPUT

```
Terminal — Node.js Execution

PS C:\Users\vinay\OneDrive\Desktop\full-stack-web-development> node observation/tasks/task6/q5_functions_vs_classes.js
Function Output (Area): 50

Class Objects:
Student: Alice | Roll No: 101 | Branch: CSE
Student: Bob | Roll No: 102 | Branch: ECE
Student: Charlie | Roll No: 103 | Branch: IT
```

```
Terminal — Node.js Execution

PS C:\Users\vinay\OneDrive\Desktop\full-stack-web-development> node observation/tasks/task6/q6_nodejs_modules.js
=== 1. OS MODULE ===
Platform: win32 | Architecture: x64 | Total Memory: 15.84 GB

=== 2. PATH MODULE ===
Joined Path: C:\Users\vinay\OneDrive\Desktop\full-stack-web-development\data\report.pdf
File Name: report.pdf | File Extension: .pdf

=== 3. FS MODULE ===
File written successfully.
File content read: Hello from Node.js FS module!
```

```
Terminal — Node.js Execution

PS C:\Users\vinay\OneDrive\Desktop\full-stack-web-development> node observation/tasks/task6/q8_express_middleware.js
Server running at http://localhost:3000
[LOG] 10:30:15 AM | Method: GET | URL: /
[LOG] 10:30:22 AM | Method: GET | URL: /about
[LOG] 10:30:45 AM | Method: GET | URL: /students
```

```
Terminal — Node.js Execution

PS C:\Users\vinay\OneDrive\Desktop\full-stack-web-development> node -e "const { Student } = require('./q5_functions_vs_classes'); Student
const student = Student('Alice', 101, 'CSE');
      ^
TypeError: Class constructor Student cannot be invoked without 'new'
    at Object.<anonymous> (C:\Users\vinay\OneDrive\Desktop\full-stack-web-development\observation\task6\q5_test.js:1:17)
    at Module._compile (node:internal/modules/cjs/loader:1256:14)

FAIL [TC-04]: ES6 Classes enforce strict instantiation using the 'new' keyword.
```

10. RESULT

Thus, the concepts of ES6 classes, built-in Node modules, NPM configurations, and Express middleware were successfully implemented and verified.